SOFTWARE ENGINEERING COURSE · PROJECT 32

Online Auction System

Complete project plan: requirements, design, implementation, testing and a day-by-day schedule to the presentation

Prepared Fri 30 Oct 2026 · Presentation Thu 5 Nov 2026 · Stack: MongoDB · Express · React · Node.js

At a glance

Item	Plan
Goal	A working online auction website, plus its requirements, UML design models and an automated test suite. These cover the three course requirements: **analyse & design**, **modular implementation**, **validation with test cases**.
Stack	**MongoDB** (database) · **Express** + **Node.js** (REST API) · **React** with Vite (user interface) · **Jest + Supertest + mongodb-memory-server** (backend tests) · **Vitest + React Testing Library** (UI tests).
Design idea	Layered backend: routes → middleware → controllers → **services** → models. Every business rule lives in the services, so the rules can be tested without a browser or server.
Scope	11 functional requirements, 10 business rules, 12 use cases, 13 API endpoints, 38 automated test cases + 5 manual checks.
Timeline	6 days: design over the weekend, backend Sun–Mon, UI Tue, polish and rehearse Wed, present Thu.
Deliverables	Source code (GitHub), README, SRS, 5 UML diagrams, API doc, test case document with results, coverage report, slides, backup demo video.

Start tonight (Fri 30 Oct) — about 2 hours

- ☐ Agree on team roles (Section 8.2) and create a GitHub repository; add every member.
- ☐ Start MongoDB: `docker run -d --name auction-mongo -p 27017:27017 mongo:8` (Docker is already installed on this laptop).
- ☐ Create the folder structure from Section 6.1 and push it.
- ☐ Copy Section 2 of this plan into `docs/SRS.md` as the first draft of the requirements.
- ☐ Everyone reads Sections 2–4 so the whole team understands the same design.

Contents

1	Project overview and scope	**7**	Testing plan and test cases
2	Requirements analysis	**8**	Schedule, team roles and Git workflow
3	Use cases	**9**	Setup guide (commands)
4	System design (architecture, class, data, state and sequence models)	**10**	Presentation and demo script
5	REST API design	**11**	Risks and stretch goals
6	Implementation plan (modules, key code, conventions)	**12**	Final deliverables checklist

1 Project overview and scope

The system lets registered users list items for auction and bid on other users' items. Each auction has a starting price, an optional hidden reserve price, a minimum bid increment and an end time. When the end time passes, the system closes the auction automatically. The highest bid wins if it meets the reserve price.

In scope (must be done by Wednesday)	Out of scope (mention as future work)
• Registration, login, JWT sessions, roles (user / admin) • Create, edit, cancel, browse and search auctions • Bidding with all business rules, safe under simultaneous bids • Automatic opening and closing of auctions, winner selection • My bids / my listings pages, admin panel • Auction page refreshes the current bid every 3 seconds • Automated tests + coverage report	• Real payments and shipping • Image file upload (use an image URL instead) • Email / SMS notifications • Proxy (automatic) bidding • WebSocket real-time push (polling is enough) • Cloud deployment (optional stretch goal)

2 Requirements analysis

2.1 Actors

Actor	Description
Guest	Visitor who is not logged in. Can browse and view auctions and register.
Bidder	Registered user placing bids. Any registered user can bid.
Seller	Registered user listing items. The same account can be a seller and a bidder.
Admin	Moderator who can view users and cancel any auction.
Scheduler	System timer that opens scheduled auctions and closes expired ones.

2.2 Functional requirements

ID	Requirement	Priority
FR-01	A guest can register with name, email and password.	Must
FR-02	A user can log in and log out. A login returns a JWT valid for 1 day.	Must
FR-03	Anyone can browse active auctions, search by keyword and filter by category.	Must
FR-04	The auction page shows the current bid, minimum next bid, time left and bid history.	Must
FR-05	A user can create an auction: title, description, category, image URL, starting price, reserve price, minimum increment, start time and end time.	Must
FR-06	A seller can edit or cancel their own auction only before the first bid.	Should
FR-07	A user can place a bid that follows business rules BR-03 to BR-07.	Must
FR-08	At the end time the system closes the auction and picks a winner (BR-08).	Must
FR-09	A user can see their bids (winning / outbid / won / lost) and their listings with results.	Must
FR-10	An admin can list users and cancel any auction that is not yet closed.	Should
FR-11	The auction page refreshes the current bid automatically every 3 seconds.	Should

2.3 Business rules

These rules are the core of the system. Each one becomes code in a service and at least one test case.

ID	Rule
BR-01	Starting price ≥ ₹1, minimum increment ≥ ₹1, and reserve price is either 0 (no reserve) or ≥ starting price.
BR-02	End time must be after start time. Duration is between 1 minute (for demos) and 30 days.
BR-03	The first bid must be ≥ the starting price.
BR-04	Every later bid must be ≥ current bid + minimum increment.
BR-05	A seller cannot bid on their own auction.
BR-06	Bids are accepted only while the status is ACTIVE and the current time is before the end time.
BR-07	The current highest bidder cannot outbid themselves.
BR-08	At the end time: if there is at least one bid and the top bid ≥ reserve, the status becomes SOLD, and the winner and final price are recorded. Otherwise the status becomes UNSOLD.
BR-09	A seller may edit or cancel only before the first bid. An admin may cancel any auction before it closes.
BR-10	Bids are never edited or deleted. They form the audit trail.

2.4 Non-functional requirements

ID	Category	Requirement
NFR-01	Security	Passwords are hashed with bcrypt and never returned by the API. Protected routes need a valid JWT. All input is validated on the server.
NFR-02	Consistency	Two simultaneous bids can never both succeed against the same auction state.
NFR-03	Performance	Bid and page requests respond in under 500 ms on a local machine.
NFR-04	Usability	Clear error messages (for example "Minimum bid is ₹1,100"). Layout works on a 375 px wide phone screen.
NFR-05	Maintainability	Layered modules, one responsibility per module, ESLint + Prettier, README with setup steps.
NFR-06	Testability	≥ 80% line coverage on the services/ folder. All tests run with a single command.
NFR-07	Reliability	The closing job can run again safely (it is idempotent). Times are stored in UTC.

3 Use cases

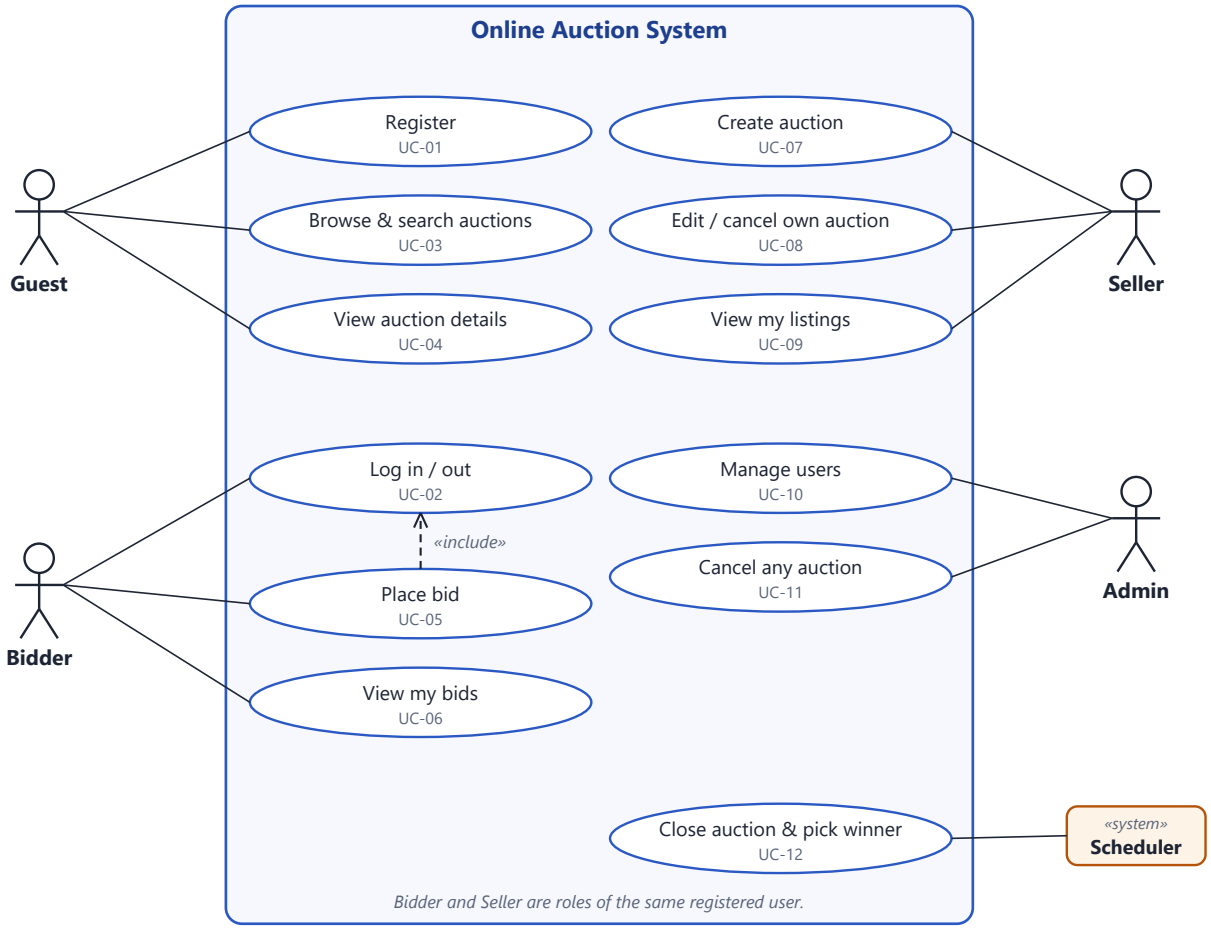


Figure 1 — Use case diagram

ID	Use case	Actor(s)	Requirement
UC-01	Register	Guest	FR-01
UC-02	Log in / out	Bidder, Seller, Admin	FR-02
UC-03	Browse & search auctions	Guest, Bidder	FR-03
UC-04	View auction details	Guest, Bidder	FR-04, FR-11
UC-05	Place bid	Bidder	FR-07
UC-06	View my bids	Bidder	FR-09
UC-07	Create auction	Seller	FR-05
UC-08	Edit / cancel own auction	Seller	FR-06
UC-09	View my listings & results	Seller	FR-09
UC-10	Manage users	Admin	FR-10
UC-11	Cancel any auction	Admin	FR-10
UC-12	Close auction & pick winner	Scheduler	FR-08

3.1 Detailed use case: UC-05 Place bid

Field	Description
Primary actor	Bidder
Preconditions	The bidder is logged in. The auction is ACTIVE and its end time has not passed.
Trigger	The bidder types an amount on the auction page and clicks **Place bid**.
Main flow	1. The system shows the current bid and the minimum next bid. 2. The bidder enters an amount ≥ the minimum and submits. 3. The system validates the input and checks BR-03 to BR-07. 4. The system saves the bid atomically and updates the current bid and highest bidder. 5. The system confirms. Other viewers see the new bid within 3 seconds.
Alternate flows	3a. Amount below the minimum → rejected; the message shows the minimum allowed. 3b. Bidder is the seller → rejected (403). 3c. Auction ended or cancelled → rejected; the page shows the final status. 3d. Bidder is already the highest bidder → rejected. 4a. Another bid was saved first (race) → rejected with "You were outbid — new minimum is …".
Postconditions	A Bid document exists. The auction's currentBid = amount, highestBidder = bidder, and bidCount is increased by 1.
Business rules	BR-03, BR-04, BR-05, BR-06, BR-07, BR-10

4 System design

4.1 Architecture

The React client talks to the Express API over REST/JSON. Inside the API, each request passes down through fixed layers. Only the **services** layer knows the business rules. That gives three benefits you can point to in the presentation: rules are in one place, each layer can be replaced on its own, and the services can be tested directly.

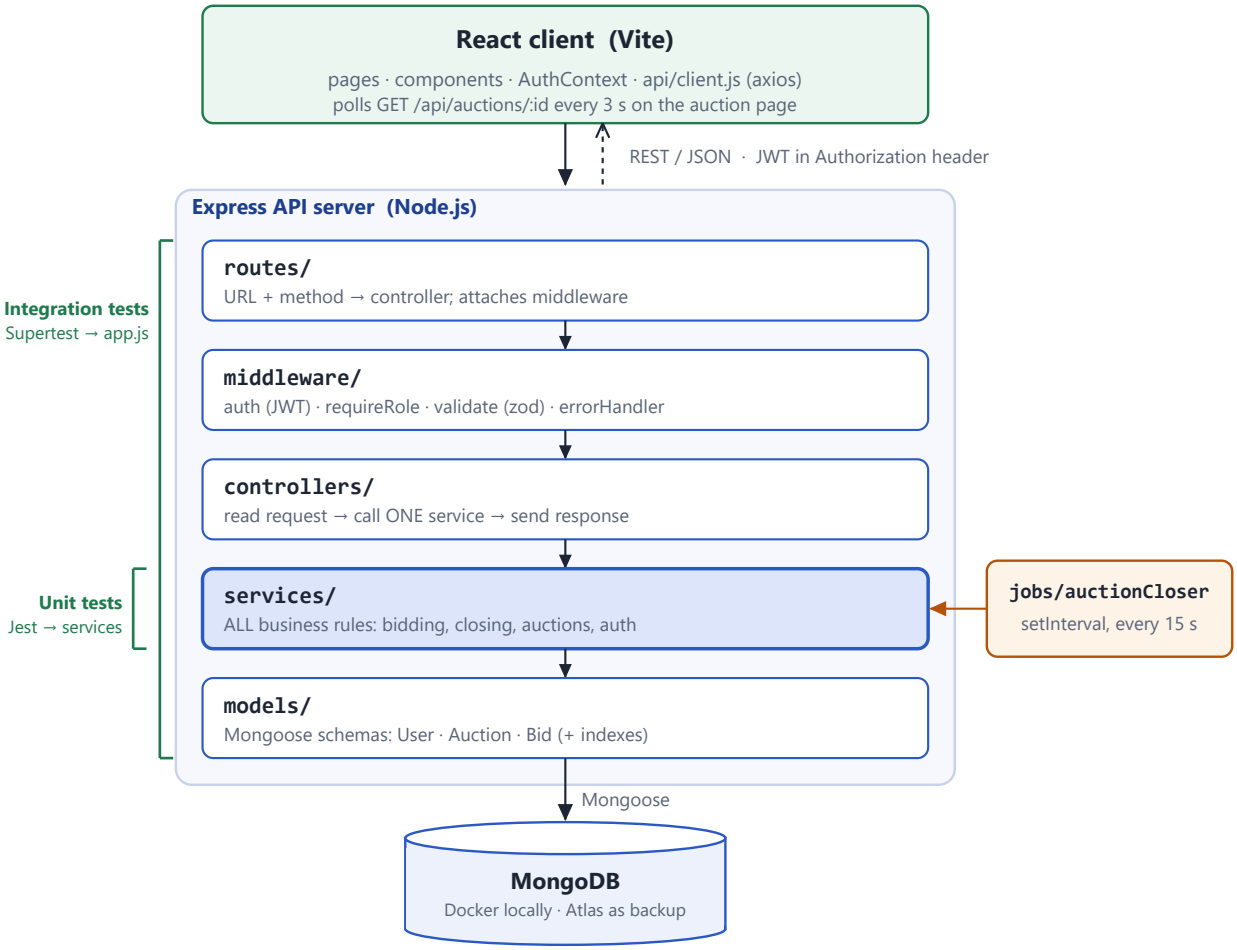


Figure 2 — Layered architecture and where each kind of test plugs in

4.2 Class diagram

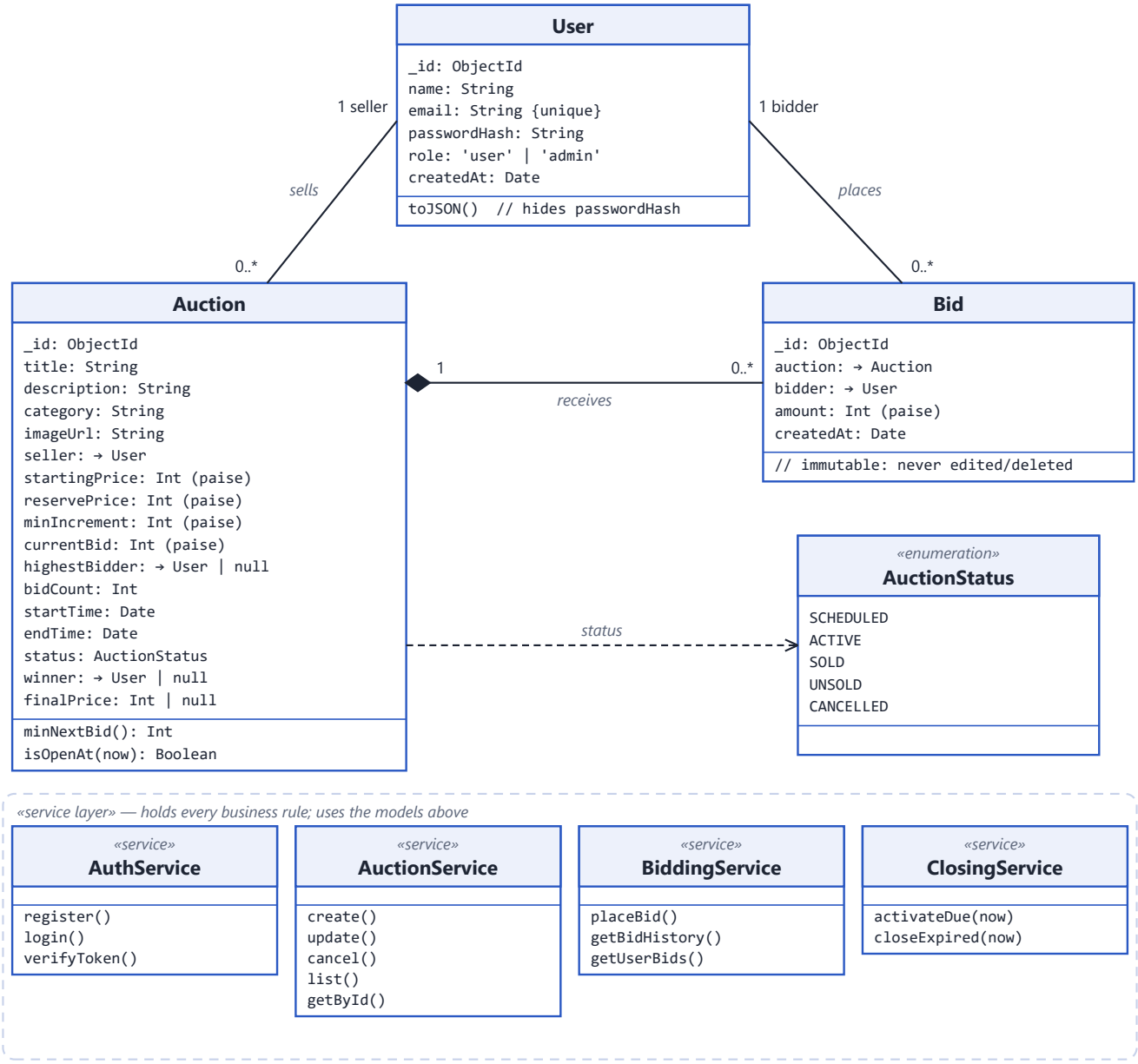


Figure 3 — Domain classes (Mongoose models) and the service layer

4.3 Data model (MongoDB collections)

Collection	Key fields	Indexes / constraints
users	name, email, passwordHash, role, createdAt	email unique; role is 'user' or 'admin'
auctions	title, description, category, imageUrl, seller, startingPrice, reservePrice, minIncrement, currentBid, highestBidder, bidCount, startTime, endTime, status, winner, finalPrice	{ status, endTime } for the closing job; text index on title + description for search; all prices are integers in paise
bids	auction, bidder, amount, createdAt	{ auction, amount: -1 } for bid history; { bidder, createdAt: -1 } for My bids

Copying `currentBid`, `highestBidder` and `bidCount` onto the auction lets the auction list load without reading the bids collection. `bidCount` also works as a version number for the safe-update check in Section 6.3.

4.4 Auction lifecycle (state diagram)

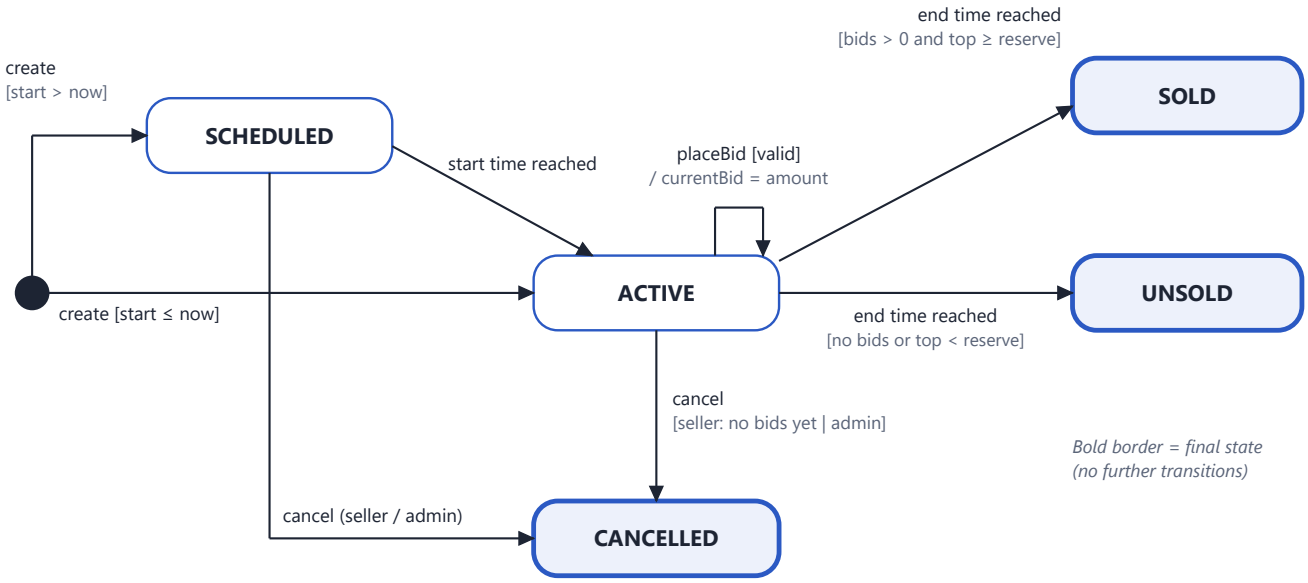


Figure 4 — State machine for an auction (tested with state-transition test cases)

4.5 Sequence diagram: Place bid

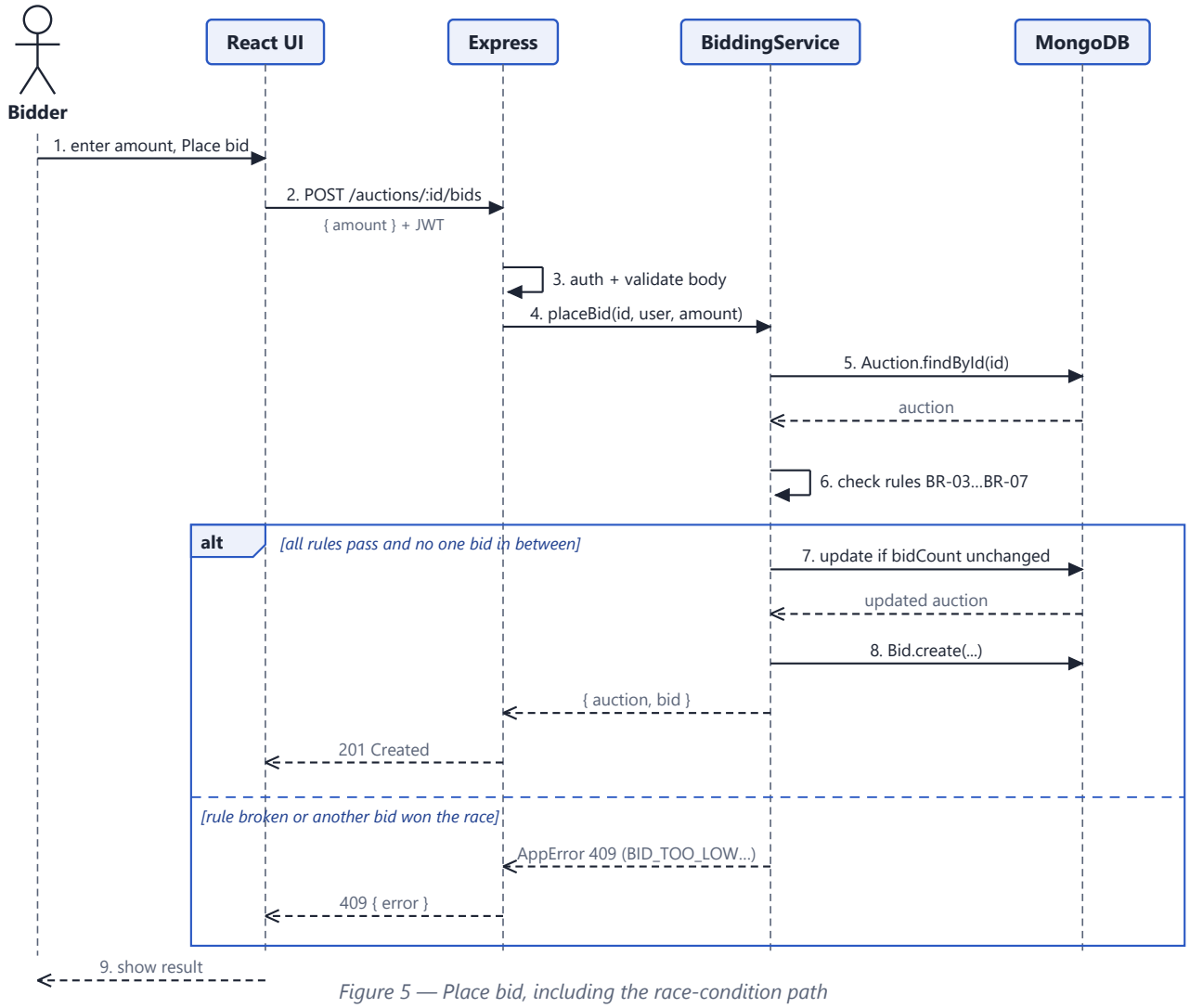


Figure 5 — Place bid, including the race-condition path

> **Tip for the slides**
> Redraw these five diagrams in draw.io or PlantUML and export PNGs into `docs/uml/`. Treat the figures in this plan as the reference version. Examiners like to see the class diagram match the code exactly.

5 REST API design

Base URL `/api`. JSON in and out. Protected routes need `Authorization: Bearer <token>`. Money is sent as integer paise.

Method	Endpoint	Auth	Purpose	Responses
`POST`	`/auth/register`	—	Create account	201 · 400, 409
`POST`	`/auth/login`	—	Log in, returns { token, user }	200 · 401
`GET`	`/auth/me`	User	Current user	200 · 401
`GET`	`/auctions`	—	List / search (?q, category, status, page)	200
`GET`	`/auctions/:id`	—	Details incl. minNextBid, timeLeft	200 · 404
`POST`	`/auctions`	User	Create auction	201 · 400
`PATCH`	`/auctions/:id`	Owner	Edit (before first bid)	200 · 403, 409

Method	Endpoint	Auth	Purpose	Responses
POST	/auctions/:id/cancel	Owner / Admin	Cancel auction	200 · 403, 409
GET	/auctions/:id/bids	—	Bid history, highest first	200 · 404
POST	/auctions/:id/bids	User	Place bid { amount }	201 · 400, 403, 404, 409
GET	/me/bids	User	My bids with WINNING / OUTBID / WON / LOST	200
GET	/me/auctions	User	My listings with results	200
GET	/admin/users	Admin	List users	200 · 403

Every error uses the same shape, so the UI can show `message` directly:

```
HTTP/1.1 409 Conflict
{ "error": { "code": "BID_TOO_LOW",
             "message": "Minimum bid is ₹1,100",
             "details": { "minAllowed": 110000 } } }
```

Status	Meaning in this project
400	Invalid input (missing field, wrong type, negative price, end time before start time)
401	Missing / invalid / expired token, or wrong login
403	Logged in but not allowed (bidding on own auction, editing someone else's auction, non-admin)
404	Auction or user not found
409	Valid request that breaks a rule given the current state (bid too low, auction not active, already highest bidder, lost the race, edit after first bid, email taken)

6 Implementation plan

6.1 Folder structure

```
online-auction-system/
├── server/
│   ├── src/
│   │   ├── config/        env.js, db.js
│   │   ├── models/        User.js, Auction.js, Bid.js
│   │   ├── services/      authService.js, auctionService.js, biddingService.js, closingService.js
│   │   ├── controllers/   authController.js, auctionController.js, bidController.js, adminController.js
│   │   ├── routes/        auth.routes.js, auction.routes.js, me.routes.js, admin.routes.js
│   │   ├── middleware/    auth.js, requireRole.js, validate.js, errorHandler.js
│   │   ├── jobs/          auctionCloser.js
│   │   ├── utils/         AppError.js, money.js
│   │   ├── app.js         builds the Express app (no listen) → imported by tests
│   │   ├── server.js      connects DB, starts job, app.listen()
│   │   └── seed.js        demo users + auctions
│   ├── tests/
│   │   ├── setup.js       in-memory MongoDB for every test file
│   │   ├── helpers.js     makeUser(), makeAuction()
│   │   ├── unit/          authService / auctionService / biddingService / closingService .test.js
│   │   └── integration/   auth.api / auctions.api / bids.api / me.api .test.js
│   ├── .env.example
│   └── package.json
├── client/
│   └── src/
│       ├── api/           client.js (axios + token), auctions.js, auth.js
│       ├── context/       AuthContext.jsx
│       ├── components/    Navbar, AuctionCard, BidForm, BidHistory, Countdown, ProtectedRoute
│       ├── pages/         Login, Register, AuctionList, AuctionDetail, CreateAuction,
│       │                  MyBids, MyListings, AdminPanel
│       └── __tests__/     BidForm.test.jsx, Countdown.test.jsx
├── docs/                  SRS.md, uml/, api.md, test-cases.md, coverage.png, slides
└── README.md
```

6.2 Module responsibilities

Layer	Does	Must NOT
`routes/`	Map URL + method to a controller; attach auth and validation middleware	Contain any logic
`middleware/`	Check JWT, check role, validate body with zod, convert errors to JSON	Contain business rules
`controllers/`	Read req, call one service function, send the response	Contain business rules or DB queries
`services/`	Apply every business rule, read/write models, throw AppError	Use req / res objects
`models/`	Schemas, indexes, small helpers such as minNextBid()	Call services
`jobs/`	Call closingService on a timer	Contain business rules
`client/api`	The only place that makes HTTP calls and attaches the token	—
`client pages`	Display data, give instant form feedback	Be the only validation (server re-checks)

6.3 Key code: placing a bid safely

This is the most important function in the project. It checks the rules and then uses a **compare-and-set** update. The write succeeds only if `bidCount` has not changed since the auction was read. If two people bid at the same moment, exactly one succeeds. This is a good point to explain in the presentation.

```
// server/src/services/biddingService.js
async function placeBid({ auctionId, bidderId, amount, now = new Date() }) {
  const auction = await Auction.findById(auctionId);
  if (!auction) throw new AppError(404, 'AUCTION_NOT_FOUND');
  if (!auction.isOpenAt(now)) throw new AppError(409, 'AUCTION_NOT_ACTIVE');          // BR-06
  if (auction.seller.equals(bidderId)) throw new AppError(403, 'OWN_AUCTION');        // BR-05
  if (auction.highestBidder?.equals(bidderId))
    throw new AppError(409, 'ALREADY_HIGHEST_BIDDER');                                // BR-07
  const minAllowed = auction.minNextBid();                                            // BR-03 / BR-04
  if (amount < minAllowed) throw new AppError(409, 'BID_TOO_LOW', { minAllowed });

  // Compare-and-set: only succeeds if nobody else bid since we read the auction
  const updated = await Auction.findOneAndUpdate(
    { _id: auctionId, status: 'ACTIVE', endTime: { $gt: now }, bidCount: auction.bidCount },
    { $set: { currentBid: amount, highestBidder: bidderId }, $inc: { bidCount: 1 } },
    { new: true }
  );
  if (!updated) throw new AppError(409, 'OUTBID_RETRY');
  const bid = await Bid.create({ auction: auctionId, bidder: bidderId, amount });  // BR-10
  return { auction: updated, bid };
}

// models/Auction.js
auctionSchema.methods.minNextBid = function () {
  return this.bidCount === 0 ? this.startingPrice : this.currentBid + this.minIncrement;
};
```

6.4 Key code: closing auctions

```
// server/src/services/closingService.js
async function activateDue(now = new Date()) {
  await Auction.updateMany({ status: 'SCHEDULED', startTime: { $lte: now } }, { status: 'ACTIVE' });
}
async function closeExpired(now = new Date()) {
  const expired = await Auction.find({ status: 'ACTIVE', endTime: { $lte: now } });
  for (const a of expired) {
    const sold = a.bidCount > 0 && a.currentBid >= a.reservePrice;                    // BR-08
    await Auction.updateOne(
      { _id: a._id, status: 'ACTIVE' },            // filter on status → safe to run twice
      sold ? { status: 'SOLD', winner: a.highestBidder, finalPrice: a.currentBid }
           : { status: 'UNSOLD' });
  }
}
// jobs/auctionCloser.js → setInterval(() => activateDue().then(() => closeExpired()), 15000)
```

6.5 Conventions everyone follows

- **Money is stored as integer paise** (₹1,100 → 110000). This avoids floating-point bugs. Format it only in the UI.
- **Time is injectable.** Every service that depends on time takes `now` as a parameter, so tests can "jump" past an end time without waiting.
- **Errors:** services throw `new AppError(status, CODE, details)`. One `errorHandler` middleware turns every error into the JSON shape in Section 5.
- **app.js does not call listen()**. Supertest imports the app directly; server.js starts it for real.
- **CommonJS** (`require`) on the server keeps Jest working with zero configuration.
- **Secrets** live in `.env` (never committed). Commit `.env.example` instead.
- ESLint + Prettier in both apps. Run `npm run lint` before every pull request.

7 Testing plan and test cases

7.1 Strategy

Level	What is tested	Tool	Approx. count
Unit (U)	Service functions with an in-memory MongoDB: every business rule and state transition	Jest + mongodb-memory-server	~22
Integration (I)	Full HTTP request → response through routes, middleware and the database	Supertest on app.js	~14
Component (C)	React components behave correctly on their own	Vitest + React Testing Library	2–4
Manual (M)	End-to-end flows in two browsers, layout, live refresh	Checklist in 7.5	5

Test design techniques used (name them on the slides):

Code	Technique	Where it is used
EP	Equivalence partitioning	Valid vs. invalid input classes: email format, roles, auction status
BVA	Boundary value analysis	Exactly the minimum bid vs. one paisa below; price 0 vs. 1; password length 7 vs. 8
ST	State-transition testing	Every arrow in Figure 4, plus illegal transitions (bidding on SOLD / CANCELLED)
DT	Decision table	Closing outcome (table below)
NEG	Negative / security testing	No token, wrong role, someone else's auction
CON	Concurrency testing	Two bids fired at the same moment with Promise.allSettled

Decision table for closing an auction (BR-08):

Rule	R1	R2	R3
Condition: at least one bid?	No	Yes	Yes
Condition: top bid ≥ reserve?	—	No	Yes
Action: final status	UNSOLD	UNSOLD	SOLD
Action: winner / final price set?	No	No	Yes
Covered by	TC-31	TC-32	TC-33

7.2 Test cases

Level: U = unit, I = integration, C = component. Amounts are in rupees for readability (stored as paise). In `docs/test-cases.md` add **Actual result** and **Pass/Fail** columns and fill them on Wednesday.

ID	Area	Scenario / input	Expected result	Tech.	Lvl
TC-01	Auth	Register with valid name, email, 8+ char password	201; user saved; no passwordHash in response	EP	I
TC-02	Auth	Register with an email that already exists	409 EMAIL_TAKEN	EP	I
TC-03	Auth	Password of 7 characters	400 validation error	BVA	U
TC-04	Auth	Password of exactly 8 characters	Accepted	BVA	U
TC-05	Auth	Login with correct credentials	200 with JWT and user	EP	I
TC-06	Auth	Login with wrong password	401 INVALID_CREDENTIALS	NEG	I
TC-07	Auth	Call POST /auctions without a token	401	NEG	I
TC-08	Auth	Normal user calls GET /admin/users	403	NEG	I
TC-09	Auction	Create valid auction starting now	201; status ACTIVE	EP	I
TC-10	Auction	Starting price ₹0	400	BVA	U
TC-11	Auction	Starting price ₹1 (smallest valid)	Created	BVA	U
TC-12	Auction	Reserve price lower than starting price	400	EP	U
TC-13	Auction	End time equal to or before start time	400	BVA	U
TC-14	Auction	Start time in the future	Status SCHEDULED	ST	U
TC-15	Auction	GET /auctions?q=camera	Only ACTIVE auctions matching "camera"	EP	I
TC-16	Auction	Seller edits title before any bid	200; title changed	ST	I
TC-17	Auction	Seller edits after one bid	409 HAS_BIDS	ST	U
TC-18	Auction	Admin cancels active auction; another user tries the same	Admin: CANCELLED; other user: 403	NEG	I
TC-19	Bidding	First bid = starting price (₹1,000)	Accepted; currentBid ₹1,000	BVA	U
TC-20	Bidding	First bid = ₹999.99	409 BID_TOO_LOW, minAllowed ₹1,000	BVA	U
TC-21	Bidding	Current ₹1,000, increment ₹100, bid ₹1,100	Accepted	BVA	U
TC-22	Bidding	Same as TC-21 but bid ₹1,099.99	409 BID_TOO_LOW	BVA	U
TC-23	Bidding	Seller bids on own auction	403 OWN_AUCTION	NEG	U
TC-24	Bidding	Highest bidder bids again	409 ALREADY_HIGHEST_BIDDER	EP	U
TC-25	Bidding	Bid 1 second after end time (before the job runs)	409 AUCTION_NOT_ACTIVE	BVA	U
TC-26	Bidding	Bid on a CANCELLED auction	409 AUCTION_NOT_ACTIVE	ST	U
TC-27	Bidding	Bid on a non-existent auction id	404	NEG	I

ID	Area	Scenario / input	Expected result	Tech.	Lvl
TC-28	Bidding	Amount negative, text, or missing	400	EP	I
TC-29	Bidding	Two users bid the same valid amount at the same moment	Exactly one accepted, one 409	CON	U
TC-30	Bidding	GET /auctions/:id/bids after 3 bids	Sorted highest first; bidder names, no emails	EP	I
TC-31	Closing	Expired auction with no bids	UNSOLD; no winner	DT	U
TC-32	Closing	Top bid ₹2,000, reserve ₹2,500	UNSOLD	DT	U
TC-33	Closing	Top bid ₹2,600, reserve ₹2,500	SOLD; winner and finalPrice set	DT	U
TC-34	Closing	SCHEDULED auction when start time passes	Becomes ACTIVE	ST	U
TC-35	Closing	Run closeExpired twice	Second run changes nothing	ST	U
TC-36	My bids	User outbid on A, leading on B, won C, lost D	OUTBID, WINNING, WON, LOST	EP	I
TC-37	UI	BidForm with min ₹1,100; type 1,050	Submit disabled; shows "Minimum ₹1,100"	BVA	C
TC-38	UI	Countdown with end time in the past	Shows "Ended"	BVA	C

7.3 Example tests

```
// tests/setup.js — fresh in-memory database for every test run
const mongoose = require('mongoose');
const { MongoMemoryServer } = require('mongodb-memory-server');
let mongo;
beforeAll(async () => { mongo = await MongoMemoryServer.create(); await mongoose.connect(mongo.getUri()); });
afterEach(async () => {
  for (const c of Object.values(mongoose.connection.collections)) await c.deleteMany({});
});
afterAll(async () => { await mongoose.disconnect(); await mongo.stop(); });
```

```
// tests/unit/biddingService.test.js
test('TC-22: bid one paisa below the minimum is rejected', async () => {
  const { auction, bob, carol } = await makeAuction({ startingPrice: 100000, minIncrement: 10000 });
  await placeBid({ auctionId: auction.id, bidderId: bob.id, amount: 100000 });
  await expect(placeBid({ auctionId: auction.id, bidderId: carol.id, amount: 109999 }))
    .rejects.toMatchObject({ code: 'BID_TOO_LOW' });
});

test('TC-29: two simultaneous bids, only one wins', async () => {
  const { auction, bob, carol } = await makeAuction({ startingPrice: 100000 });
  const results = await Promise.allSettled([
    placeBid({ auctionId: auction.id, bidderId: bob.id,   amount: 120000 }),
    placeBid({ auctionId: auction.id, bidderId: carol.id, amount: 120000 }),
  ]);
  expect(results.filter(r => r.status === 'fulfilled')).toHaveLength(1);
});

test('TC-33: reserve met → SOLD', async () => {
  const { auction, bob } = await makeAuction({ reservePrice: 250000, endsInMs: 60000 });
  await placeBid({ auctionId: auction.id, bidderId: bob.id, amount: 260000 });
  await closeExpired(new Date(Date.now() + 120000));          // jump 2 minutes ahead
  const after = await Auction.findById(auction.id);
  expect(after).toMatchObject({ status: 'SOLD', finalPrice: 260000 });
});
```

```
// tests/integration/bids.api.test.js
test('TC-07: placing a bid without a token returns 401', async () => {
  const res = await request(app).post(`/api/auctions/${id}/bids`).send({ amount: 100000 });
  expect(res.status).toBe(401);
});
```

7.4 Coverage

Run `npm run test:cov` in `server/`. Target ≥ 80% lines on `services/` and ≥ 70% overall. Take a screenshot of the summary table and the HTML report (`coverage/lcov-report/index.html`) for the slides.

7.5 Manual test checklist

ID	Steps	Expected
M-01	Open the same auction in two browsers (normal + incognito). Bid in one.	The other shows the new bid within 3 s
M-02	Watch an auction reach its end time	Page switches to SOLD / UNSOLD within ~15 s, without a reload
M-03	Logged out, click Place bid	Redirected to login, then back to the auction
M-04	Create auction with end time before start time	Inline error; nothing saved
M-05	Resize to 375 px wide	All pages usable, no horizontal scrolling

7.6 Requirements traceability matrix

Shows that every requirement is designed, built and tested. Put this on a slide; examiners look for it.

Requirement	Use case	Module(s)	Test cases
FR-01 Register	UC-01	authService, auth.routes	TC-01 – TC-04
FR-02 Login / JWT	UC-02	authService, middleware/auth	TC-05 – TC-08
FR-03 Browse / search	UC-03	auctionService.list	TC-15
FR-04 Auction details	UC-04	auctionService.getById, BidHistory, Countdown	TC-30, TC-37, TC-38
FR-05 Create auction	UC-07	auctionService.create, validate	TC-09 – TC-14
FR-06 Edit / cancel	UC-08	auctionService.update / cancel	TC-16 – TC-18
FR-07 Place bid	UC-05	biddingService.placeBid	TC-19 – TC-29
FR-08 Close auction	UC-12	closingService, jobs/auctionCloser	TC-31 – TC-35, M-02
FR-09 My bids / listings	UC-06, UC-09	biddingService.getUserBids	TC-36
FR-10 Admin	UC-10, UC-11	adminController, requireRole	TC-08, TC-18
FR-11 Auto refresh	UC-04	AuctionDetail (polling)	M-01

8 Schedule, team roles and Git workflow

8.1 Day-by-day plan

Day	Focus	Tasks	Done when
Fri 30 Oct (tonight)	Kick-off	Assign roles · create GitHub repo · start MongoDB in Docker · create folders · SRS first draft	Everyone can clone the repo; SRS.md exists
Sat 31 Oct	Design + backend skeleton	All 5 UML diagrams · finalise schemas and API table · Express app.js / server.js · DB connection · models · AppError + errorHandler · Jest setup with in-memory DB	`npm test` runs one passing test; diagrams exported to docs/uml
Sun 1 Nov	Core services	authService · auctionService · biddingService (with compare-and-set) · closingService · unit tests TC-03/04, 10–14, 17, 19–26, 29, 31–35	All unit tests green; services coverage ≥ 80%
Mon 2 Nov	API + frontend start	Routes, controllers, auth / role / validate middleware · closer job · integration tests · seed script. Frontend: Vite app, router, AuthContext, api client, Login, Register, AuctionList	Full flow works in Thunder Client / Postman; integration tests green; can log in from React
Tue 3 Nov	Frontend complete	AuctionDetail (BidForm, BidHistory, Countdown, 3 s polling) · CreateAuction · MyBids · MyListings · AdminPanel · component tests TC-37/38 · manual tests M-01 to M-05	The complete demo script (Section 10.2) works in the browser
Wed 4 Nov	Polish + rehearse	Fix bugs · README · fill test-case results · coverage screenshot · slides · record backup demo video · 2 full rehearsals · **code freeze 8 pm** (tag v1.0)	Slides final; everyone can present any slide
Thu 5 Nov	Present	Arrive early · start Docker · `npm run seed` · open browser tabs · run the tests once	Presentation delivered

Rule for the week

No new features after Tuesday night. Wednesday is only for fixing, documenting and rehearsing. A finished small system scores better than an unfinished big one.

8.2 Team roles (for 4 members; with 3, merge A and D)

Member	Role	Owns	Backup for
A	Analyst / designer / lead	SRS, UML diagrams, traceability matrix, slides, keeps the schedule	D
B	Backend core	Models, all four services, unit tests	C
C	Backend API + QA	Routes, controllers, middleware, closer job, seed, integration tests, coverage report	B
D	Frontend	All React pages and components, component tests, manual test checklist	A

Everyone reviews at least one pull request a day and must be able to explain Figures 2, 3 and 5.

8.3 Git workflow

- `main` must always run. Work on short branches: `feat/bidding-service`, `feat/auction-detail-page`, `test/closing-service`.
- Open small pull requests. One teammate reviews; merge the same day to avoid painful conflicts.
- Commit message prefixes: `feat:` `fix:` `test:` `docs:` `refactor:`.
- Optional but impressive: a GitHub Actions workflow that runs `npm test` on every push (about 15 minutes to set up).

9 Setup guide

Node.js, npm and Docker are already installed on this laptop. Run these from the project root.

```
# 1) Database
docker run -d --name auction-mongo -p 27017:27017 mongo:8
#    next time: docker start auction-mongo

# 2) Backend
mkdir server && cd server
npm init -y
npm i express mongoose bcryptjs jsonwebtoken cors dotenv zod
npm i -D jest supertest mongodb-memory-server nodemon eslint prettier
cd ..

# 3) Frontend
npm create vite@latest client -- --template react
cd client
npm i react-router-dom axios
npm i -D vitest jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event
```

server/package.json (scripts and Jest config):

```
"scripts": {
  "dev": "nodemon src/server.js",
  "start": "node src/server.js",
  "seed": "node src/seed.js",
  "test": "jest --runInBand",
  "test:cov": "jest --runInBand --coverage",
  "lint": "eslint src tests"
},
"jest": { "testEnvironment": "node", "setupFilesAfterEnv": ["./tests/setup.js"] }
```

server/.env.example:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/auction
JWT_SECRET=replace-with-a-long-random-string
JWT_EXPIRES_IN=1d
CLOSER_INTERVAL_MS=15000
CLIENT_ORIGIN=http://localhost:5173
```

Seed data (`npm run seed`): users alice (seller), bob and carol (bidders), admin. Six auctions, including one that ends 10 minutes after seeding (for the live demo; the minutes can be set with `DEMO_END_MINUTES`), one SCHEDULED, and one already SOLD. All demo passwords: `Password1`.

10 Presentation and demo script

10.1 Slide plan (about 14 minutes; trim to your time limit)

#	Slide	Content	Who	Min
1	Title	Project name, team, course	A	0.5
2	Problem & objectives	Why online auctions; the 3 course goals	A	1
3	Requirements	Actors, key FRs, business rules BR-03 to BR-08, NFRs	A	1.5
4	Use case diagram	Figure 1 + UC-05 summary	A	1
5	Architecture	Figure 2: layers and why rules live in services	B	1
6	Class + data model	Figure 3, money in paise, indexes	B	1
7	State + sequence	Figures 4 and 5; how the race condition is handled	B	1.5
8	Live demo	Section 10.2	D	3
9	Testing strategy	Levels, techniques, decision table	C	1
10	Test results	Pass count, coverage screenshot, TC-29 explained	C	1
11	Traceability	Matrix from 7.6	C	0.5
12	Wrap-up	Challenges, limitations, future work, Q&A	A	1

10.2 Live demo script (3 minutes)

About 10 minutes before your slot: `docker start auction-mongo` → `npm run seed` (this starts the 10-minute clock on the "ending soon" auction) → start server and client → log in as **alice** in a normal window, **bob** in incognito and **carol** in a second browser.

1. **Alice** creates "Vintage Camera": start ₹1,000, reserve ₹2,500, increment ₹100, ends in 3 minutes.
2. **Bob** opens it and bids ₹1,000 → accepted. Carol's screen updates within 3 s.
3. **Carol** bids ₹1,050 → rejected: "Minimum bid is ₹1,100" (BR-04, boundary test TC-22).
4. **Alice** tries to bid on her own camera → rejected (BR-05).
5. **Carol** bids ₹1,200. **Bob** bids ₹2,600.
6. Switch to the **seeded auction** that ends during the demo → it flips to SOLD with the winner shown. (Don't wait for the camera auction.)
7. Terminal: `npm run test:cov` → all green + coverage table.

> **Backup plan**
>
> Record the whole demo on Wednesday (Win + Alt + R with the Xbox Game Bar, or OBS). If Wi-Fi, Docker or the laptop misbehaves, play the video and keep talking. Also keep a MongoDB Atlas free-tier connection string in `.env.atlas` in case Docker won't start.

10.3 Likely questions and short answers

Question	Answer
Why MongoDB and not SQL?	Auction documents are self-contained and read far more often than written. Mongoose still gives a schema and validation. The trade-off: no joins and no default multi-document transactions, which compare-and-set handles.
What if two people bid at the same time?	The update only succeeds if bidCount is unchanged since the read. Exactly one succeeds; the other gets 409. TC-29 proves it.
Why separate services from controllers?	Rules are in one place, testable without HTTP, and reusable by the closer job. Controllers stay tiny.
How do you test time-based logic?	Services take a 'now' parameter, so tests pass a future time instead of waiting.
Unit vs. integration test here?	Unit tests call a service function directly. Integration tests send real HTTP requests through routes and middleware using Supertest.
Why store money in paise?	Floating point can't represent 0.1 exactly; integers are always exact.
What would you add next?	WebSockets for instant updates, proxy bidding, payments, email notifications, deployment.

11 Risks and stretch goals

Risk	Likelihood	Impact	Mitigation
Running out of time on the frontend	Medium	High	Backend finished by Monday; keep UI plain; no new features after Tuesday
Docker / MongoDB fails on demo day	Low	High	Atlas backup connection string; recorded demo video
mongodb-memory-server download fails	Medium	Medium	Run npm test once on good internet on Saturday (it downloads mongod once, ~100 MB, then caches it)
Merge conflicts between members	Medium	Medium	Small PRs merged daily; each member owns separate folders
A member is unavailable	Low	High	Every area has a named backup (Section 8.2)
Time-zone bugs with end times	Medium	Medium	Store UTC; send ISO strings; show local time only in the UI

Stretch goals (only if everything is done by Tuesday evening)

- Socket.io push updates instead of polling.
- Watchlist / favourite auctions.
- Proxy bidding: the user sets a maximum, and the system bids for them in increments.
- Deploy: backend on Render, frontend on Vercel, database on MongoDB Atlas.
- GitHub Actions running the tests on every push (cheap and impressive).

12 Final deliverables checklist

- ☐ GitHub repository with tag `v1.0`; `main` runs from a clean clone using the README.
- ☐ README: features, stack, setup steps, demo accounts, how to run the tests, screenshots.
- ☐ `docs/SRS.md`: actors, FRs, business rules, NFRs (Section 2).
- ☐ `docs/uml/`: use case, class, state, sequence and architecture diagrams (PNG + source).
- ☐ `docs/api.md`: endpoint table and error format (Section 5).
- ☐ `docs/test-cases.md`: TC-01 to TC-38 and M-01 to M-05 with actual results and Pass/Fail.
- ☐ Coverage report screenshot: services ≥ 80%.
- ☐ Traceability matrix (Section 7.6).

- ☐ Slides (12) and a backup demo video.
- ☐ Each member has rehearsed their part and can answer the questions in 10.3.

> **Summary**
> Weekend: design and core services with tests. Monday: API. Tuesday: UI. Wednesday: polish and rehearse. Keep every business rule in the services layer, test each rule at its boundaries, and use the compare-and-set bid update as your strongest technical talking point.